PRÉTRAITEMENT

Pipeline de prétraitement – du brut au dataset prêt pour le modèle

1 Chargement & exploration

hargement du dataset Pima (CSV)
Analyse initiale :

- types de variables (dtypes)
- statistiques (describe)
- aperçu des données (head)

2 Analyse

Détection de valeurs aberrantes
Présence de zéros non biologiques dans :

- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

3 Split 75/25

- Séparation train / test
- Stratification sur la variable cible

4 Zeros → NaN

Remplacement des 0 par NaN uniquement dans les colonnes médicales, puis imputation.

5 Imputation

- Médiane (sur le train uniquement) pour Glucose, BloodPressure, BMI (faible % de NaN).
- KNN Imputer (k=5) pour SkinThickness et Insulin (fort % de NaN).

6 Standardisation

avant l'imputation KNN pour équilibrer l'impact des variables dans le calcul des distances

Règle d'or : tous les fit (médiane, scaler, KNN) sont faits uniquement sur le train, puis appliqués au test → pas de data leakage

Étape 1 — Chargement du Dataset & Exploration initiale

Chargement du dataset

```
import numpy as np
import pandas as pd

url = "https://raw.githubusercontent.com/plotly/datasets/refs/heads/master/diabetes.csv"
df = pd.read_csv(url)
```

Types de variables (df.dtypes)

```
les différentes variables et leurs types:

Pregnancies                   int64
Glucose                       int64
BloodPressure                 int64
SkinThickness                 int64
Insulin                       int64
BMI                         float64
DiabetesPedigreeFunction    float64
Age                           int64
Outcome                       int64
dtype: object
```

- Variables numériques uniquement :
- int64 (entiers)
- float64 (réels)

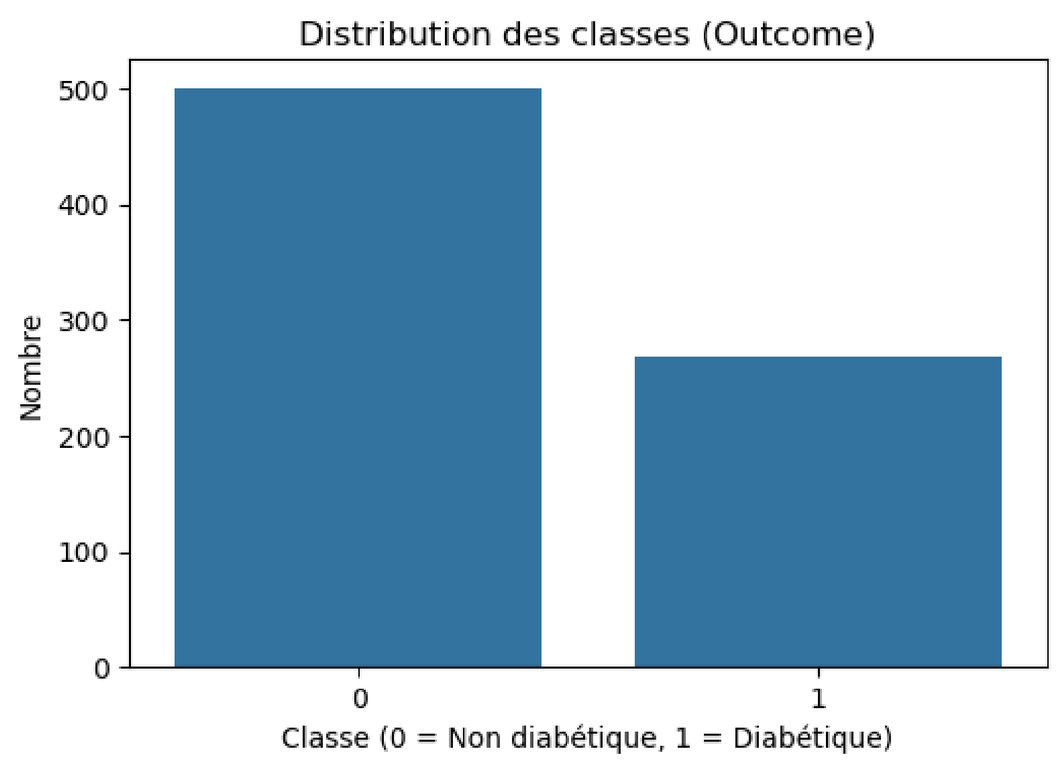
Étape 2 — Détection des Valeurs Aberrantes & Déséquilibre de classes

Calcul du % de zeros par colonne

```
Glucose : 0.65% de valeurs égales à 0
BloodPressure : 4.56% de valeurs égales à 0
SkinThickness : 29.56% de valeurs égales à 0
Insulin : 48.70% de valeurs égales à 0
BMI : 1.43% de valeurs égales à 0
```

Distribution des classes (Outcome)

Distribution des Classes — Variable Cible : Outcome



Vérification des données

```
print(f'le nombre de doublons:', df.duplicated().sum())
✓ 0.0s

le nombre de doublons: 0
```

Étape 2 — Détection des Valeurs Aberrantes & Déséquilibre de classes

Stratégie d'imputation choisie

Variable	Taux de valeurs manquantes	Méthode choisie
Glucose · BP · BMI	< 5%	Médiane train
SkinThickness	29.6%	KNN Imputer k=5
Insulin	48.7%	KNN Imputer k=5

Pourquoi la médiane plutôt que la moyenne ?

Moyenne sensible aux valeurs extrêmes (ex: Insulin max=846). Un outlier tire la moyenne → imputation biaisée.

Médiane robuste aux outliers. Si 50% des valeurs d'Insulin sont 0, la médiane n'est pas perturbée par le max=846.

Étapes 3 & 4 — Split Stratifié & Remplacement des 0 par NaN

Split des données (75% / 25%)

- **Séparation :**
  - Train : 75%
  - Test : 25%
- **Méthode utilisée :**
  - garantit que chaque split a ~65% classe 0 et ~35% classe 1, évitant un déséquilibre accidentel.

Remplacer les 0 par NaN (sur train ET test)

```
cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]

X_train[cols] = X_train[cols].replace(0, np.nan)
X_test[cols] = X_test[cols].replace(0, np.nan)
```

Pourquoi pas remplacer les 0 AVANT le split ? → Car la médiane serait calculée sur 768 obs. (train+test) au lieu de 576 obs. (train seul). Le test set aurait 'influencé' les statistiques d'imputation, ce qui constitue un data leakage.

Pourquoi split avant tout traitement ?

DATA LEAKAGE (fuite de données) : si on impute AVANT de splitter, les statistiques (médiane, KNN) sont calculées sur train+test. Le modèle 'voit' indirectement les données de test

Résultat : performance artificiellement élevée (biaisée)

Pourquoi seulement ces 5 colonnes ?

Pregnancies : 0 grossesse est biologiquement POSSIBLE → on garde
Age : 0 an n'existe pas (min=21)

DPF : Valeur réelle, pas de 0 dans la colonne

5 colonnes : Glucose, BP, BMI, Skin, Insulin : 0 = mesure non prise → NaN

Étapes 5 & 6 — Imputation par Médiane & StandardScaler

Imputation par médiane (Glucose, BP, BMI)

```
low_missing = ["Glucose", "BloodPressure", "BMI"]


for col in low_missing:
    median = X_train[col].median()
    X_train[col] = X_train[col].fillna(median)
    X_test[col] = X_test[col].fillna(median)
```

StandardScaler (AVANT le KNN Imputer)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Médiane vs KNN — quand choisir ?

Médiane (simple) si taux NaN < 5-10%. Rapide, robuste aux outliers. La distribution de la variable ne change presque pas.

KNN Imputer si taux NaN élevé (>20%). Utilise les k voisins les plus proches pour estimer la valeur manquante → imputation plus précise mais plus lente.

Pourquoi scaler AVANT le KNN Imputer ?

Le KNN Imputer calcule des **distances euclidiennes** entre observations pour trouver les k voisins.

Si Insulin (max=846) n'est pas normalisé, il **domine la distance** et les autres features sont ignorées → mauvaise imputation.

Après StandardScaler : toutes les features contribuent équitablement.

Étape 7 — KNN Imputer (SkinThickness & Insulin)

KNN Imputer

Objectif : estimer les valeurs manquantes à partir des patients similaires

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)

X_train_imputed = imputer.fit_transform(X_train_scaled)
X_test_imputed = imputer.transform(X_test_scaled)
```

Comment fonctionne KNN Imputer ?

Pour une obs. avec Insulin=NaN :

① Calcule la distance euclidienne avec toutes les autres obs. sur les features CONNUES (Glucose, BMI, Age…)
② Sélectionne les k=5 plus proches voisins qui ont une valeur d'Insulin connue
③ Impute par la **moyenne des 5 valeurs d'Insulin**

Pourquoi k=5 ?

k=5 est la valeur par défaut de sklearn, empiriquement bonne.
k trop petit (1-2) → bruit (un seul voisin outlier biaise l'imputation).
k trop grand (>20) → lissage excessif, perd la structure locale.